Lecture 28.12: Decision Tree Regressor — Continuous Partitioning & Piecewise Constants

📌 Executive Summary

Formulating Decision Trees for continuous target regression, partitioning feature space into disjoint regions, and predicting regional sample averages.

🏛️ Theoretical Formulation & Mathematical Architecture

📈 Regression Trees & Piecewise Constant Approximations

When the target supervisory response is continuous ($y \in \mathbb{R}$), the tree partitions the domain $\mathcal{X}$ into M disjoint orthogonal regions $\{R_1, R_2, \ldots, R_M\}$:

$$\mathcal{X} = \bigcup_{m=1}^{M} R_m \quad \text{where } R_j \cap R_k = \emptyset \quad \forall j \neq k$$

The model represents the global continuous response function as a linear combination of indicator basis functions:

$$\boxed{f(\mathbf{x}) = \sum_{m=1}^{M} c_m \cdot \mathbb{I}(\mathbf{x} \in R_m)}$$

📐 Mathematical Derivation of Optimal Leaf Prediction c_m

Under the classical Sum of Squared Errors (SSE) loss criterion:

$$J(c_m) = \sum_{i \in R_m} (y_i - c_m)^2$$

To find the minimizing constant c_m^*, compute the partial derivative with respect to c_m and set to zero:

$$\frac{\partial J}{\partial c_m} = -2 \sum_{i \in R_m} (y_i - c_m) = 0$$

$$\sum_{i \in R_m} y_i - |R_m| c_m = 0 \implies \boxed{c_m^* = \frac{1}{|R_m|} \sum_{i \in R_m} y_i = \bar{y}_{R_m}}$$

Thus, the optimal leaf prediction under squared error loss is strictly the **sample mean** of all training instances assigned to region R_m.

💻 Vectorized Python Implementation

```
import numpy as np
from sklearn.tree import DecisionTreeRegressor

# Generate non-linear 1D sine signal
np.random.seed(42)
X_1d = np.sort(np.random.uniform(0, 10, 80)).reshape(-1, 1)
y_1d = np.sin(X_1d).ravel() + np.random.normal(0, 0.1, 80)

reg = DecisionTreeRegressor(max_depth=3, random_state=42)
reg.fit(X_1d, y_1d)

preds = reg.predict(X_1d)
print("Unique Piecewise Constant Levels:", len(np.unique(preds)))
assert len(np.unique(preds)) <= 2**3, "Depth 3 tree cannot have more than 8 distir
```

🔬 Key Engineering Takeaways & Invariants

1. **Axis-Aligned Partitioning**: Decision trees recursively split feature space along orthogonal boundaries without requiring feature scaling.
2. **Impurity Optimization**: Split selection deterministically maximizes impurity reduction (Information Gain for classification, Variance Reduction for regression).

3. **Generalization Safeguards**: Regularization via pre-pruning (`max_depth` , `min_samples_leaf`) and post-pruning (`ccp_alpha`) prevents noisy leaf memorization.